

GEO4171: Instructions on the R-software

Contents:

GEO4171: Instructions on the R-software.....	1
The R-software.....	2
About R.....	2
Online information on R.....	3
How to install R.....	3
Extra packages (Add-on packages).....	3
Installation of extra packages.....	4
Installation in RStudio:.....	4
Installation from R using the menu:.....	4
Installation from R using commands (this works on any platform):.....	4
Loading of packages.....	5
Exploratory Data Analysis.....	6
Getting started.....	6
Help.....	6
Changing the working directory.....	6
Loading packages and macros.....	6
Exploratory data analysis.....	7
Loading the data.....	7
Generating monthly values and getting annual maxima (AMS).....	7
Looking for linear trends.....	8
Fitting AMS to the GEV distribution.....	9

The R-software

To be able to apply the methods covered by the lectures we advise you to use the statistical software R. You will need this for Assignment 1.

About R

R is a language and environment for statistical computing and graphics. R is similar to the S and S-Plus language and environment which was developed at Bell Laboratories. There are some important differences, but much code written for S runs unaltered under R.

R provides a wide variety of statistical (linear and nonlinear modelling, classical statistical tests, time-series analysis, classification, clustering, etc) and graphical techniques, and is highly extensible.

R is available as Free Software under the terms of the Free Software Foundation's GNU General Public License in source code form. It compiles and runs on a wide variety of UNIX platforms and similar systems (including FreeBSD and Linux), Windows and MacOS.

R is an integrated suite of software facilities for data manipulation, calculation and graphical display. It includes

- an effective data handling and storage facility,
- a suite of operators for calculations on arrays, in particular matrices,
- a large, coherent, integrated collection of intermediate tools for data analysis,
- graphical facilities for data analysis and display either on-screen or on hardcopy, and
- a well-developed, simple and effective programming language which includes conditionals, loops, user-defined recursive functions and input and output facilities.

The term "environment" is intended to characterize it as a fully planned and coherent system, rather than an incremental accretion of very specific and inflexible tools, as is frequently the case with other data analysis software.

R, like S, is designed around a true computer language, and it allows users to add additional functionality by defining new functions. Much of the system is itself written in the R dialect of S, which makes it easy for users to follow the algorithmic choices made. For computationally-intensive tasks, C, C++ and Fortran code can be linked and called at run time. Advanced users can write C code to manipulate R objects directly.

Many users think of R as a statistics system. We prefer to think of it of an environment within which statistical techniques are implemented. R can be extended (easily) via packages. There are about eight packages supplied with the R distribution and many more are available through the CRAN family of Internet sites covering a very wide range of modern statistics.

Online information on R

The R-website provides online [documentation](#) and [introduction](#) to R. Additional manuals, the software and further information can be found on the R-website. Try the example session in [Appendix A](#) to get a feeling of the possibilities within R.

How to install R

It is recommended to download and install [Rstudio](#).

Alternatively, download and install R via the [R-website](#).

Under ‘Download and Install R’ you choose the operating system for which you want to download the program. Binaries in this context means that you will get a program file which is already compiled for a specific operating system and can be run directly after the installation. This should be suitable for most users. Under special circumstances, advanced users might also download the source code and compile it themselves.

Under Windows, the program is then installed by running the installation program you download.

Further information on installation of R for different operating systems is given in the manual ‘R Installation and Administration’(R-admin.pdf).

Extra packages (Add-on packages)

In R, all functions (and also some example data sets) are stored in *packages*. A function of a package can only be used, if:

1. **the package is installed on the computer**, i.e. it is available in a folder on the hard disk. A folder containing packages is called a library, and the standard library in R is ‘R_Home/library’ (e.g. ‘C:\Program Files\R\R-2.8.1\library’).
2. **the package is loaded**.

The R program contains several standard packages which are installed during the installation of R. Some of these standard packages are also automatically loaded every time the program is started.

However, there are many extra packages (add-on packages) with additional functions and these have to be installed and loaded ‘by hand’. **For this course**, we will need the extra packages:

plotrix	Various plotting functions including polar plots
evir	Extreme Values in R
nsRFA	Includes l-moment calculations
lubridate	Date- and time-related functions

A complete overview of all packages available is given on <http://cran.r-project.org/> under ‘Packages’.

In the following two sections, a short description of how to install and load extra packages is given. Additional information on installation of packages is found in [R Installation and administration](#) (Chapter 6) and on loading of packages in [An introduction to R](#) (Chapter 13).

Installation of extra packages

Installation in RStudio:

1. In the menu ‘Tools’ choose ‘Install package(s) ...’. Then you can specify the name of the package to install.

Installation from R using the menu:

1. In the menu ‘Packages’ choose ‘Install package(s) ...’
2. A pop-up window ‘CRAN mirror’ opens. Select a CRAN mirror, i.e. a server which is close to your location (e.g. ‘Denmark’), and click ‘ok’. The ‘CRAN mirror’ window will not open if you have already selected a mirror for the session. Then you can directly proceed to the next bullet point.
3. In the pop-up window ‘Packages’ select the package you want to install (e.g. ‘evd’) and click ‘ok’. The package will now be installed.

Installation from R using commands (this works on any platform):

1. Installation with internet connection

Choose a CRAN mirror

Type the command: **chooseCRANmirror()** and select a CRAN mirror in the following pop up window. If no pop up window appears, you have to use the command **chooseCRANmirror(graphics = FALSE)** and specify the CRAN mirror with a number from the list that is shown after the command is given.

Install the package

Type the command: **install.packages("packagename")** with ‘packagename’ specifying the name of the package to install, e.g. ‘evd’.

If you do not have administrator rights you cannot install the packages in the standard library. You then have to install them in a folder that you have created yourself. Use then the command: **install.packages("packagename", lib= "path")**, with ‘path’ specifying the path to the folder where the package should be installed. In this case, you also have to specify the new library path afterwards with **.libPaths("path")** where ‘path’ is the path used in the installation above. Otherwise the new installed package will not be found if it should be loaded (see below).

2. Installation without internet connection

2.1 Download the Windows binary package from <http://cran.r-project.org/> under ‘Packages’. For this of course an internet connection is necessary. When downloading for example the package ‘evd’, the current file name will be ‘evd_2.1-7.zip’

2.2 Install the package

Type the command: **install.packages("source_path/ evd_2.1-7.zip", repos=NULL)** with ‘source_path’ specifying the path to the directory, where the downloaded zip-package is located (in this example the ‘evd’ package).

If you do not have the administrator rights you can install the packages in a new directory analogue to 1.2:

install.packages("source_path/evd*.zip", repos=NULL, lib= "path")**

with ‘path’ specifying the path to the new library directory. Add the new library path afterwards with **.libPaths("path")**.

Loading of packages

Some of the installed standard packages are loaded automatically when R is started. To check, which packages are currently loaded, use the command

search()

All additional packages that should be used during an R session, must be loaded every time the program is started. This is done with the command

library("packagename")

To check, which packages are installed and available for loading, use the library command without additional argument:

library()

Exploratory Data Analysis

Getting started

Help

To start the online help type: `help.start()`

To get information on a specific function, type: `help(function_name)`

For some short symbols the argument must be put in quotes, e.g.: `help("+")`

Changing the working directory

At the start of a session you should **change the working directory**. As a command line, write: `setwd("E:/Baerbar/Documents/UiO/GE04171/Assignments")`

Replace the path name given in red with your own path name. When you specify the path to your working directory, you should use the forward slash '/' and you should re-type the " symbols in the command window.

Alternatively in R, use the menu 'File' → 'Change dir ...' to a new folder and in Rstudio use the menu "Session" → 'Set working directory' → 'Choose directory'

In the work-directory you put the files you want to work with. You can also save your workspace and history at the end of a session.

The **workspace and history are saved automatically at the end of a session** (command `q()`), if you answer with 'yes'. The **workspace** contains all objects (e.g. variables and functions) that you have defined during your session and is saved in the file '**.RData**'. The **history** contains all commands that you have used during a session and is saved in '**.Rhistory**'. If you start R the next time by double clicking on '.R.Data', R will be started with the workspace and history from the previous session. Also the working directory will be automatically set to this folder. Alternatively, you can save the workspace and history via the menu with self-defined names ('File' → 'Save Workspace'; 'File' → 'Save History'). However, if you start R then by double clicking on this 'Name.RData' file, the history is not automatically loaded but must be loaded via the menu ('File' → 'Load History').

To see the commands you have run in the R console so far in the current session, you can also type the command `history()`. Then all commands you have run will be displayed in the "History" tab, usually in the top right corner of RStudio.

Loading packages and macros

To load the additional packages type:

```
library(plotrix)
```

```
library(nsRFA)
```

```
library(evir)
```

```
library(lubridate)
```

Alternatively, you can load the packages via the menu: 'Packages' → 'Load package'.

The packages must be loaded each time you start a new R-session!

To load the functions that are stored in the file 'GEO4171_macro.R'

```
source("GEO4171_macro.R")
```

If you save your workspace at the end of the session and restore the workspace in the next session, you don't have to load the macros again in the next session.

Exploratory data analysis

Loading river discharge data

To read time series data from a file, create a plain text file with data organised in columns. The first line can be used for column headers. Any sign, including tab and space, can be used for separating the columns. **Note that the first column should contain dates in the format 'YYYYMMDD', 'YYYYMM' or 'YYYY' depending on the time resolution of the data.** You can have as many columns as you want so that you can put several variables in one file.

To read file downloaded from sildre.nve.no, type:

```
daily_data_raw <- read.table("62.5.0-Vannføring-dogn-v1.csv",
skip=2, sep=";", dec = ",")
dd = as.integer(substr(daily_data_raw[,1],1,4))*10000+
as.integer(substr(daily_data_raw[,1],6,7))*100+
as.integer(substr(daily_data_raw[,1],9,10))
daily_data = cbind(dd,daily_data_raw[,2])
correction = cbind(dd,daily_data_raw[,3])
quality = cbind(dd,daily_data_raw[,4])
```

In order to assign a NoData value (i.e. missing) to negative values, type:

```
daily_data[!is.na(daily_data[,2]) & (daily_data[,2] < 0),2] <- NA
```

Generating monthly values and getting annual maxima (AMS)

Before proceeding, check whether your discharge data begin in January on the first recorded year, and that they end in December on the last year. Otherwise, delete all data for the first (or last) year.

In order to obtain monthly averaged time series (note that you never type [Macro]):

```
monthly_data <- daily_to_monthly(daily_data) [Macro]
```

In order to obtain climatological averages for each of the 12 months:

```
monthly_average <- monthly_to_monthly_average(monthly_data) [Macro]
```

In order to obtain climatological monthly averages for the first and second half of the data:

```
monthly_average_first <-
monthly_to_monthly_average(monthly_data[1:(dim(monthly_data)[1]/2),])
[Macro]
```

```
monthly_average_second <-
monthly_to_monthly_average(monthly_data[(1+dim(monthly_data)[1])/2]:
(dim(monthly_data)[1]),])
[Macro]
```

To make a time series plot of the daily data, type (after the command, you need to hit ‘enter’ another time to confirm the page change, or click into the graphics window):

```
see_data(daily_data, 365) [Macro]
```

Before extracting AMS data, clean up your time series by removing years with a significant number of missing daily values (e.g. > 5%), especially if data are missing in periods where floods generally occur at your station. Then use the following to extract AMS data:

```
amsdata <- get_ams(daily_data) [Macro]
```

To add information about correction and quality of flood data:

```
amsdata$correction = correction[correction[,1]%in%amsdata$samdates, 2]  
amsdata$quality = quality[quality[,1]%in%amsdata$samdates, 2]
```

And print them on the screen:

```
print(cbind(amsdata$samdates, amsdata$correction))  
print(cbind(amsdata$samdates, amsdata$quality))
```

The correction mark should be 3 (or 0) for ams data

The quality mark should be 0. If there has been correction, please make a table summarizing the corrected flood values. You should consider excluding these years from your dataset.

To plot the annual maximum data:

```
plot(floor(amsdata$maxdates), amsdata$ams, type='p', xlab="Year", ylab="Streamflow (m3/s) ")
```

Type='l' gives a line plot; type = 'p' gives a point plot. Further plotting types are given in chapter 12 of ‘An introduction to R’ (R-intro.pdf, p. 63).

To plot time series of daily data together with AMS data:

```
dates_daily = as.Date(as.character(daily_data[,1]), format="%Y%m%d")  
plot(dates_daily, daily_data[,2], type="l", xlab="Date",  
ylab=parse(text="Discharge (m^3/s)"))  
dates_ams = as.Date(as.character(amsdata$samdates), format="%Y%m%d")  
points(dates_ams, amsdata$ams, col=2)
```

To plot the seasonal variation in streamflow type

```
plot_seasonal(monthly_average) [Macro]
```

To plot the seasonal variation in streamflows for the first and second half of the data:

```
plot_seasonal(monthly_average_first, monthly_average_second)
```

To plot the seasonal variation in floods (flood rose):

```
plot_season_floods(daily_data) [Macro]
```

You can also for example add a title and save the figure using R code. There are several ways to do this ([e.g. this link](#)). Here is an example of how to save the flood rose figure as a jpeg:

```
jpeg("floodrose_figure.jpg", quality=100)  
plot_season_floods(daily_data) [Macro]  
title("Flood Rose")
```



```
dev.off()
```

Fitting AMS to the GEV distribution

You will use a Bayesian approach for fitting a distribution to the data. The approach uses non-informative priors for the location and scale parameters, whereas the shape parameter has an *a priori* standard deviation of 0.2 and expected value of zero. This prior helps to get reasonable estimates. Note that in the call below a variance parameter of 0.01 is also set for the “proposal distribution” of shape parameter. This is a parameter for the MCMC algorithm, and the chosen values ensures that the estimation of the shape parameter is more robust.

```
Myfit <- BayesianMCMC (xcont= amsdata$ams, confint = c(0.025, 0.975),  
dist = "GEV", apriori = myprior, varparameters=c(NA, NA, 0.01))
```

where “confint” is a vector with the cumulated probability values defining the boundaries of the confidence interval. Here the boundaries are the 0.025 and 0.975 quantiles, so 95% confidence intervals will be estimated. ‘myprior’ is the prior distribution for the GEVparameters with non-informative priors on location and scale parameters and a normal distribution with mean 0 and standarddeviation 0.2 on the shape parameter.

To get the parameter values with the highest posterior likelihood (best estimate):
Myfit\$parametersML

And you get the three parameter values; location (par 1), scale (par 2) and shape (par 3):

par 1	par 2	par 3
42.61335236	10.74759973	-0.07301056

To get the 95% confidence intervals of the parameter estimates:

```
get_parameter_ci(Myfit,c(0.025,0.975))
```

[Macro]

and you will get the lower and upper bounds for each parameter:

	Par1	Par2	Par3
0.025	40.07560	9.03399	-0.22392659
0.975	45.35355	13.58243	0.06796826

The return periods for which estimates are provided are given by:

```
Myfit$returnperiods
```

To get the best estimate of return levels (design floods) for the associated return periods given by **Myfit\$returnperiods**:

```
Myfit$quantilesML
```

And to get the lower and upper boundaries for a 95% credibility interval:

```
Myfit$intervals
```

Use spline interpolation to get the flood estimates for a specific return time, e.g. 100 years
splinefun(Myfit\$returnperiods,Myfit\$quantilesML)(100)

To get several flood estimates corresponding to several return times

```
splinefun(Myfit$returnperiods,Myfit$quantilesML)(c(100,200,1000))
```

which returns three values for 100, 200 and 1000 year return times, respectively

Similarly, use spline interpolation to compute the lower bound of the 95% confidence interval of flood estimates corresponding to several return times
`splinefun(Myfit$returnperiods,Myfit$intervals[1, ])(c(100,200,1000))`
 and the upper bound
`splinefun(Myfit$returnperiods,Myfit$intervals[2, ])(c(100,200,1000))`

Return levels plots:

`plot(Myfit,xlab="Return period (years)", ylab = "Flood size (m3/s)")`

And diagnostic plots of the fit are given by:

`gev.diag.bayes(Myfit)`

If the layout looks incorrect (square frame, axis labels not visible, lines outside the plot frames, etc.), redo the plot after running

`dev.off()`

Design peak flood at target site

You need two last steps to get the design flood at your target site.

- 1: Adjust the design flood from your donor catchment to your target site
- 2: At the target site, adjust the daily design flood to peak design flood

Adjusting design flood from donor to target site

Depending on how far away your donor catchment is from your target site, use the following approaches:

If the target site is directly upstream or downstream of the donor site and the overlapping catchment area is large (e.g. 80-90 % overlap), you can use the ratio of catchment areas ($A_{\text{target}} / A_{\text{donor}}$). However, if there are large gradients in runoff, it would be more correct to use the ratios of mean annual runoff as a scaling factor ($Q_{\text{mean, target}} / Q_{\text{mean, donor}}$) where Q_{mean} is given in m^3/s . The mean annual runoff can be extracted for the normal period 1991-2020 (available from nevina.nve.no)

If the donor site is further away, the best approach is to use the ratio of mean annual flood (m^3/s): ($Q_{\text{MAF, target}} / Q_{\text{MAF, donor}}$).

Daily to peak flood

The first step is to assess if the target site has dominating snow melt or rain floods. It can be done using this equation

$$p = \frac{1}{1 + \exp\left(-\left(24.08 - 3.66 \cdot \exp(W_{\text{Nov}}/100) - 4.28 \cdot \ln(P_{\text{Jun}})\right)\right)}$$

Where it is a rain flood dominated catchment if $p > 0.5$ and snow melt dominated if $p < 0.5$. W_{Nov} (mm/month) is average sum of snow melt and rain in November and P_{Jun} is mean precipitation in June.

If the target site is mainly affected by rain floods, the correction factor to get from daily to hourly flood peak can be calculated as:

$$\varphi = 1 + \exp \left(\begin{array}{l} 0.582 + 0.000035 * (F_s)^3 \\ + 0.00402 * A_{SE} * \ln(W_{Med4Max}) \\ - 0.671 * \log_{10}(A) - 0.862 * \sqrt{A_{SE}} \end{array} \right)$$

Where F_s (degrees) is mean slope , A_{SE} is effective lake percentage (%), A (km²) is catchment area, and $W_{med4Max}$ (mm/day) is the median for 4 day rain and snow melt.

If your target site is mainly dominated by snowmelt:

$$\varphi = 1 + \exp(-1.38 + 0.979 * \log_{10}(Q_N) - 0.668 * \log_{10}(A) - 0.858 * \sqrt{A_{SE}}) \quad (51)$$

Where Q_N (l/s/km²) is mean annual runoff